

Syntactic Language Extension via an Algebra of Languages and Transformations

Jacob Andersen

Department of Computer Science
Aarhus University; Aarhus, Denmark

Claus Brabrand

IT University of Copenhagen
Copenhagen, Denmark

September 9, 2009

Abstract

We propose an algebra of languages and transformations as a means for extending languages syntactically. The algebra provides a layer of high-level abstractions built on top of *languages* (captured by context-free grammars) and *transformations* (captured by constructive catamorphisms).

The algebra is *self-contained* in that any term of the algebra specifying a transformation can be *reduced* to a catamorphism, before the transformation is run. Thus, the algebra comes “for free” without sacrificing the strong safety and efficiency properties of constructive catamorphisms.

The entire algebra as presented in the paper is implemented as the Banana Algebra Tool which may be used to syntactically extend languages in an incremental and modular fashion via algebraic composition of previously defined languages and transformations. We demonstrate and evaluate the tool via several kinds of extensions.

1 Introduction and Motivation

We propose an algebra of 16 operators on languages and transformations as a *simple*, *incremental*, and *modular* way of specifying *safe* and *efficient* syntactic language extensions through algebraic composition of previously defined languages and transformations.

Extension is *simple* because we base ourselves on a well-proven and easy-to-use formalism for well-typed syntax-directed transformations known as *constructive catamorphisms*. These transformations are specified relative to a source and

a target language which are defined via context-free grammars (CFGs). Catamorphisms have previously been studied and proven sufficiently expressive as a means for extending a large variety of programming languages via transformation [5, 6, 7]. Hence, the main focus of this paper lies not so much in addressing the expressiveness and which transformations can be achieved as on showing how algebraic combination of languages and transformations results in highly modular and incremental language extension. *Incremental* and *modular* means that any previously defined language or transformations may be composed algebraically to form new languages and transformations. *Safety* means that the tool statically guarantees that the transformations always terminate and only map syntactically legal input terms into syntactically legal output terms; *Efficiency* means that any transformation is guaranteed to run in linear time (in the size of input and generated output).

An important property of the algebra which is built on top of catamorphisms is that it is “self-contained” in the sense that any term of the algebra may be *reduced* to a constant catamorphism, at compile-time. This means that all high-level constructions offered by the algebra (including composition of languages and transformations) may be dealt with at compile-time, before the transformations are run, without sacrificing the strong safety and efficiency guarantees.

Everything presented in the paper has been implemented in the form of The Banana Algebra Tool which, as argument, takes a transformation term of the algebra which is then analyzed for safety and *reduced* to a constant catamorphism which may subsequently be run to transform an input program.

The tool may be used for many different transformation purposes, such as transformation between different languages (e.g., for translating Java programs into HTML documentation in the style of JavaDoc or for prototyping lightweight domain-specific language compilers), transforming a given language (e.g., the CPS transformation), format conversion (e.g., converting BibTeX to BibTeXML). However, in this paper we will focus on *language extension* for which we have the following usage scenarios in mind: 1) Programmers may extend existing languages with their own macros; 2) Developers may embed domain-specific languages (DSLs) in host languages; 3) Compiler writers may implement only a small core and specify the rest externally; and 4) Developers or teachers may define languages incrementally by stacking abstractions on top of each other. We will substantiate these usage claims in Section 6.

The approach captures the niche where full-scale compiler generators as outlined in Section 7 are too cumbersome and where simpler techniques for syntactic transformation are not expressive or safe enough, or do not have sufficient support for incremental development.

Our contributions include the design of an algebra of languages and transformations for incremental and modular syntactic language extension built on top of catamorphisms; a proof-of-concept tool and implementation capable of working with concrete syntax; and an evaluation of the algebraic approach.

2 Catamorphisms

A *catamorphism* (aka., *banana* [16]) is a generalization of the *list folding* higher-order function known from functional programming languages which processes a list and builds up a return value. However, instead of working on lists, it works on any inductively defined datatype. Catamorphisms have a strong category theoretical foundation [16] which we will not explore in this paper. A catamorphism associates with each constructor of the datatype a *replacement evaluation function* which is used in a transformation. Given an input term of the datatype, a catamorphism then performs a recursive descent on the input structure, effectively deconstructing it, and applies the replacement evaluation functions in a bottom-up fashion recombining intermediate results to obtain the final output result.

Many computations may be expressed as catamorphisms. As an example, let us consider an inductively defined datatype, `list`, defining non-empty lists of numbers:

$$\text{list} = \text{Num } \mathbb{N} \mid \text{Cons } \mathbb{N} * \text{list}$$

The sum of the values in a list of numbers may easily be defined by a catamorphism, by replacing the `Num`-constructor by the identity function on numbers ($\lambda n.n$) and the `Cons`-constructor by addition on numbers ($\lambda(n,l).n+l$), corresponding to the following recursive definition:

$$\begin{aligned} \llbracket \text{Num } n \rrbracket &= n \\ \llbracket \text{Cons } n \ l \rrbracket &= n + \llbracket l \rrbracket \end{aligned}$$

One of the main advantages of catamorphisms is that recursion over the structure of the input is completely separated from the construction of the output. In fact, the recursion is completely determined from the input datatype and is for that reason often only specified implicitly. Since the sum catamorphism above maps terms of type `list` to natural numbers $\mathbb{N}$, it may be uniquely identified with its replacement evaluation functions; in this case with a replacement evaluation function for the `Num`-constructor of type $\mathbb{N} \rightarrow \mathbb{N}$ and a replacement function of type $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ for `Cons`). Catamorphisms are often written in the so-called banana brackets “ $\llbracket \dots \rrbracket$ ” [16]:

$$\llbracket \lambda n.n \ , \ \lambda(n,l).n+l \ \rrbracket$$

2.1 Constructive Catamorphisms

Constructive catamorphisms are a restricted form of catamorphisms where only output-typed *reconstructors* are permitted as replacement evaluator functions. Reconstructors are just constructor terms from (possibly different) inductively defined datatypes wherein the arguments to the constructive catamorphism may be used. For instance, we can transform the lists into binary trees of the `tree` datatype:

`tree = Nil | Leaf N | Node N * tree * tree`

using a constructive catamorphism:

$$\begin{aligned} \llbracket \text{Num } n \rrbracket &= \text{Leaf } n \\ \llbracket \text{Cons } n \ l \rrbracket &= \text{Node } n \ (\text{Nil}) \ \llbracket l \rrbracket \end{aligned}$$

Although very simple, capable of trivial recursion only, we claim that this kind of constructive catamorphisms provide a basis for programming language extension. We shall investigate this claim in the following section.

2.2 Safety and Efficiency

Constructive catamorphisms have a lot of interesting properties; they can be statically verified for *syntactic safety*, are guaranteed to terminate, and to run in linear time.

A constructive catamorphism, c , is *typed* with a source language, l_s , and a target language, l_t , as in “ $l_s \rightarrow l_t$ ”. The languages can be given either as a *datatype* (at the abstract syntactic level) as above, or as a *CFG* (at the concrete syntactic level). A constructive catamorphism is said to be *syntactically safe* if it only produces syntactically valid output terms, $\omega_t \in \mathcal{L}(l_t)$, given syntactically valid input terms, $\omega_s \in \mathcal{L}(l_s)$:

$$\forall \omega \in \mathcal{L}(l_s) \Rightarrow c(\omega) \in \mathcal{L}(l_t)$$

In addition to a *language typing* ($l_s \rightarrow l_t$), we also need a *nonterminal typing*, τ , which for each of the nonterminals of the input language specifies onto which nonterminal of target language they are mapped.

If we name the source and target languages of the above example **Lists** and **Trees** respectively, the language typing then becomes “**Lists** $\rightarrow$ **Trees**” and the nonterminal typing, τ , is “[**list** $\rightarrow$ **tree**]”. (The reason for the angled bracket convention is that there may be multiple nonterminals in play, in which case multiple mappings are written as a comma separated list inside the brackets.)

In order to verify that a catamorphism, $\llbracket l_s \rightarrow l_t \ [\tau] \ c \rrbracket$ is syntactically safe, one simply needs to check that each of the catamorphism’s reconstructor terms (e.g., “**Node** n (**Nil**) $\llbracket l \rrbracket$ ”) are valid syntax, assuming that each of its argument usages (e.g., $\llbracket l \rrbracket$) are valid syntax of the appropriate type (in this case l has source type **list** which means that $\llbracket l \rrbracket$ has type $\tau(\text{list}) = \text{tree}$). We refer to [1] for a formal treatment of how to verify syntactic safety.

Constructive catamorphisms are highly efficient. Asymptotically, they run in linear time in the size of the input and output: $O(|\omega| + |c(\omega)|)$.

3 Language Extension

We will now illustrate—using deliberately simple examples—how constructive catamorphisms may be used to extend programming languages and motivate

the idea of programming language extension. To this end, let us consider the core λ -Calculus (untyped, without constants) whose syntactic structure may be defined by the following datatype:

$$\text{exp} = \text{Var id} \mid \text{Lam id} * \text{exp} \mid \text{App exp} * \text{exp}$$

In the following, we will investigate how to extend the λ -Calculus using catamorphisms; in particular, we will look at two well-known extensions, namely that of numerals and booleans.

3.1 Extension: Numerals

A common extension of the core λ -Calculus is that of *numerals*; the calculus is extended with a construction representing *zero*, and unary constructors representing the *successor* and *predecessor* of a numeral. These constructions may be combined to represent any natural numbers in unary encoding and for performing numeric calculations. The syntax of the calculus is then extended to the language, LN:

$$\begin{aligned} \text{exp} = & \text{Var id} \mid \text{Lam id} * \text{exp} \mid \text{App exp} * \text{exp} \mid \\ & \text{Zero} \mid \text{Succ exp} \mid \text{Pred exp} \end{aligned}$$

We will now show how a catamorphism may be used to transform the extended language, LN, into the core λ -Calculus, L, using a basic encoding of numerals which represents *zero* as the identity function $(\lambda z.z)$, and a number n as follows:

$$\overbrace{\lambda s . \lambda s . \dots \lambda s .}^{n \text{ lambdas}} \overbrace{\lambda z . z}^{\text{zero}}$$

There are many other possible encodings of numerals, including the more commonly used Church numeral representation, but the choice of encoding is not of primary interest here, so we will just use the simpler alternative to illustrate the point. We can now extend the λ -Calculus with numerals as a constructive catamorphism of type “LN $\rightarrow$ L [exp $\rightarrow$ exp]”:

$$\begin{aligned} \llbracket \text{Var } V \rrbracket &= \text{Var } \llbracket V \rrbracket \\ \llbracket \text{Lam } V \ E \rrbracket &= \text{Lam } \llbracket V \rrbracket \ \llbracket E \rrbracket \\ \llbracket \text{App } E_1 \ E_2 \rrbracket &= \text{App } \llbracket E_1 \rrbracket \ \llbracket E_2 \rrbracket \\ \llbracket \text{Zero} \rrbracket &= \text{Lam } z \ (\text{Var } z) \\ \llbracket \text{Succ } E \rrbracket &= \text{Lam } s \ \llbracket E \rrbracket \\ \llbracket \text{Pred } E \rrbracket &= \text{App } \llbracket E \rrbracket \ (\text{Lam } z \ (\text{Var } z)) \end{aligned}$$

The first three rules just trivially recurse through the input structure producing an identical output structure. Zero becomes the identity function, successor adds a “lambda s” in front of the encoding of the argument, and predecessor peels off one lambda by applying it to the identity function (note that the predecessor of zero is thus consequently defined as zero). This will, for instance, map `Succ Zero` to its encoding `Lam s (Lam z (Var z))`.

3.2 Other Extensions

Similarly, the core λ -Calculus may easily be extended with *booleans* (via nullary constructors `True` and `False`, and a ternary `If`) yielding a syntactically extended language `LB` which could then be transformed to the core λ -calculus by a constructive catamorphism with typing “`LB -> L [exp -> exp]`”:

$$\begin{aligned} \llbracket \text{True} \rrbracket &= \text{Lam } a \text{ (Lam } b \text{ (Var } a)) \\ \llbracket \text{False} \rrbracket &= \text{Lam } a \text{ (Lam } b \text{ (Var } b)) \\ \llbracket \text{If } E_1 \ E_2 \ E_3 \rrbracket &= \text{App (App } \llbracket E_1 \rrbracket \llbracket E_2 \rrbracket) \llbracket E_3 \rrbracket \end{aligned}$$

Note that we have omitted the three lines of “identity transformations” for variables, lambda abstraction, and application.

Along similar lines, the λ -Calculus could be further extended with *addition*, *multiplication*, *negation*, *conjunction*, *lists*, *pairs*, and so on, eventually converging on a full-scale programming language. To substantiate the claim that this forms an adequate basis for language extension, we have extended the λ -Calculus towards a language previously used in teaching functional languages; “Fun” (cf. Section 6).

4 Algebra of Languages and Transformations

Investigating previous work on syntactic macros and transformations [5, 6, 7] has revealed an interesting and recurring phenomenon in that macro extensions follow a certain pattern. The first hint in this direction is the effort involved in the first three lines of the constructive catamorphisms which are there merely to specify the “identity transformation” on the core λ -Calculus. That effort could be alleviated via explicit language support.

In fact, every such language extension can be broken into the same five ingredients (some of which are *languages*, some of which are *transformations*), depicted in Figure 1: i) a core language that is to be extended (e.g., the λ -Calculus); ii) a language extension of that language¹ (e.g., the extension with numerals); iii) an identity transformation on the core language; iv) a transformation that maps the extended language to the core language; and v) a notion of “addition” of the identity transformation and the small transformation of the language extension to the core language.

4.1 The Algebra

The five ingredients above can be directly captured by five algebraic operators. First, cases i) and ii) correspond to a *constant language* operator which may be modeled by a context-free grammar (with “named productions” for attaching transformations). Second, case iii) corresponds to a constant transformation which may be given as an output-typed constructive catamorphism, c , typed with the source and target languages of the transformation (and a nonterminal

¹Note that we refer to the extended language as excluding the core language.

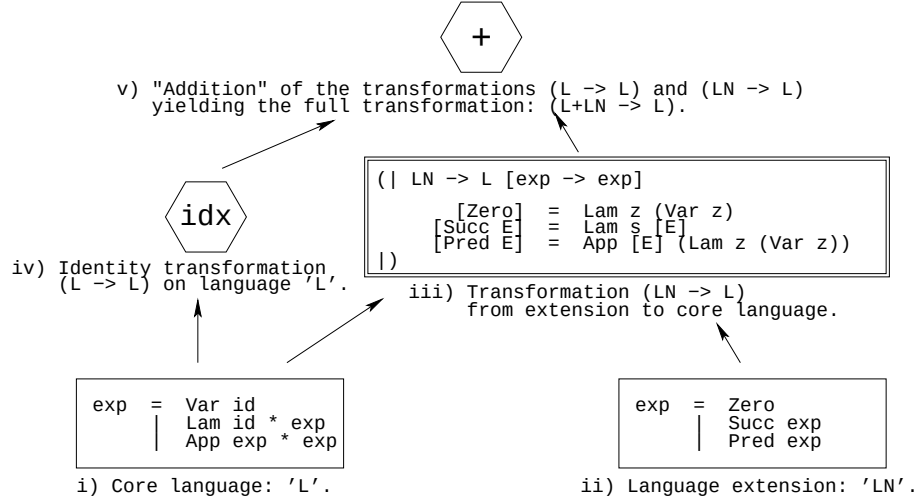


Figure 1: Common pattern in language extension (here extending the λ -Calculus with numerals.)

typing, τ). Third, case iv) corresponds to an operator taking a language l and turning it into the identity transformation ($l \rightarrow l$) on that language. Fourth, a notion of addition on transformations, taking two transformations $l_s \rightarrow l_t$ and $l'_s \rightarrow l'_t$ yielding a transformation: $(l_s \oplus_l l'_s) \rightarrow (l_t \oplus_l l'_t)$ where “ $\oplus_l$ ” is addition on languages. Language addition is defined as the *union* of the individual productions (transformation addition as the union of the catamorphic reconstructors), which in both cases ensure that addition is *idempotent*, *symmetric*, *associative*, and *commutative*. For a formal definition of addition on languages and transformations, we refer to [1].

Note that with these operations, it is very easy to obtain a transformation combining *both* the extension of numerals and booleans; simply “add” the two transformations.

Although the above algebraic operations are enough to make all the extensions of the previous chapter, we would like to motivate a couple more algebraic operators on languages and transformations. Note that even though the design, and choice of operators arose through an iterative process, we have tried to divide and categorize the motivations for the constructions into two categories; operators accommodating respectively *modular* and *incremental* language extension. The complete syntax for the algebra is presented in Figure 2. (The rules for *language constants*, *transformation constants*, *language addition*, *transformation addition*, and *identity transformations* are numbered L1, X1, L4, X4, and X6, respectively.) Of course, it is possible to add even more operators to the algebra; however, the ones we have turn out to be sufficient to conveniently extend the λ -Calculus incrementally all the way to the Fun programming language. These ideas are pursued in the remainder of the paper which also includes

L	$\rightarrow_{L1}$	l	X	$\rightarrow_{X1}$	$\langle L \rightarrow L [\tau] c \rangle$
	$\rightarrow_{L2}$	v		$\rightarrow_{X2}$	w
	$\rightarrow_{L3}$	$L \setminus L$		$\rightarrow_{X3}$	$X \setminus L$
	$\rightarrow_{L4}$	$L + L$		$\rightarrow_{X4}$	$X + X$
	$\rightarrow_{L5}$	$\text{src} (X)$		$\rightarrow_{X5}$	$X \circ X$
	$\rightarrow_{L6}$	$\text{tgt} (X)$		$\rightarrow_{X6}$	$\text{idx} (L)$
	$\rightarrow_{L7}$	$\text{let } v=L \text{ in } L$		$\rightarrow_{X7}$	$\text{let } v=L \text{ in } X$
	$\rightarrow_{L8}$	$\text{letx } w=X \text{ in } L$		$\rightarrow_{X8}$	$\text{letx } w=X \text{ in } X$

(a) Algebra of languages (L)... (b) ...and transformations (X).

Figure 2: Syntax of the algebra.

an evaluation of the whole algebraic approach. For a formal specification of the semantics of the algebra, see the Appendix (for a specification of the underlying languages and transformations, see [1]).

4.2 Modular language extension

In order to permit modular language development and separate each of the ingredients in a transformation, we added *local definition* mechanism via the standard **let-in** functional programming local binder construction. Thus, we add to the syntax of both languages and transformations; *variables* (Figure 2, rules L2 and X2) and *local definitions* (Figure 2, rules L7, and X7).

In practice, it turns out to be useful to also be able to define (local) *transformations* while specifying *languages*; and, orthogonally, to define (local) *languages* while specifying *transformations*. Hence, we add the local definitions L8 and X8 to Figure 2.

4.3 Incremental language extension

Transformations are frequently specified incrementally in terms of previously defined languages and transformations. To accommodate such use we added a means for designating the *source* and *target* languages of a transformation along with a means for *restricting* a language and a transformation (i.e., restricting the source language of a transformation). By restriction, we take “ $L_1 \setminus L_2$ ” to yield a language identical to L_1 , but where all productions also mentioned by name in L_2 have been eliminated. (The operators mentioned are listed as rules L5, L6, L3, and X3 of Figure 2.)

Also, transformations are frequently expressed via intermediate syntactic constructions for either simplicity or legibility. For instance, notice how two of the catamorphic reconstructors in the transformation of Section 3.1 both use the identity lambda abstraction **Lam z (Var z)**. Here, one could specify this transformation incrementally, by using an intermediary language, LI, enriched with identity as an explicit nullary construction:

$$\text{exp} = \text{Var id} \mid \text{Lam id} * \text{exp} \mid \text{App exp} * \text{exp} \mid \text{Id}$$

Although on such a small example, there is little to gain in terms of simplicity and/or legibility, it illustrates the general principle of incremental language extension. The transformation (“LN $\rightarrow$ L”) can now be simplified to “**ln2li**: LN $\rightarrow$ LI”:

$$\begin{aligned} \llbracket \text{Zero} \rrbracket &= \text{Id} \\ \llbracket \text{Succ } E \rrbracket &= \text{Lam } s \llbracket E \rrbracket \\ \llbracket \text{Pred } E \rrbracket &= \text{App } \llbracket E \rrbracket (\text{Id}) \end{aligned}$$

Which is subsequently composed with the tiny transformation that desugares the identity-enriched language to the core λ -Calculus, “**li2l**: LI $\rightarrow$ L”:

$$\llbracket \text{Id} \rrbracket = \text{Lam } z (\text{Var } z)$$

Not surprisingly, when we do this experiment using the tool, the transformation “**li2l** $\circ$ **li2ln**” produces the exact same transformation as the directly specified constant transformation in Section 3.1. To enable such incremental development, we added *composition* as an operator on transformations (cf. Figure 2, rule X5).

Note that none of the operators go beyond the expressivity of constructive catamorphisms in that any language term can be statically reduced to a context-free grammar; and any transformation term to a catamorphism.

An important advantage of an algebraic approach is that several algebraic laws hold which give rise to simplifications (e.g., “ $L + L \equiv L$ ”, “ $L_1 + L_2 \equiv L_2 + L_1$ ”, “ $L_1 + (L_2 + L_3) \equiv (L_1 + L_2) + L_3$ ”, “ $\text{src}(\text{id}(L)) \equiv L$ ”) to mention but a few. (For a formal specification of the reduction and semantics of the operators, see the Appendix.)

5 Tool and Implementation

In order to validate the algebraic approach, we have implemented everything in the form of The Banana Algebra Tool which we have used to experiment with different forms of language extensions.

5.1 Abstract vs. Concrete Syntax

A key issue in building the tool was the choice of whether to work with *abstract* or *concrete* syntax. Everything we have presented so far has been working exclusively on the abstract syntactic level. For practical usability of the tool, however, it turns out to be more convenient to work on the concrete syntax. Note that because of the addition operators of the algebra, it is important that particular choice of parsing algorithm be closed under union.

Figure 3 illustrates the difference between using abstract and concrete syntax for specifying transformations. Figure 3(a) depicts a fragment of a grammar for

Exp.or	: Exp1 " " Exp ;	Stm.repeat =	Stm.repeat =
.exp1	: Exp1 ;	Stm.do(<1>,	'do <1> while (!(<2>));' ;
Exp1.and	: Exp2 "&&" Exp1 ;	Exp.exp1(	
.exp2	: Exp2 ;	Exp1.exp2(	
Exp2.add	: Exp3 "+" Exp2 ;	Exp2.exp3(	
.exp3	: Exp3 ;	Exp3.exp4(	
...		Exp4.exp5(	
Exp7.neg	: "!" Exp8 ;	Exp5.exp6(	
.exp8	: Exp8 ;	Exp6.exp7(	
Exp8.par	: "(" Exp ")" ;	Exp7. <u>neg</u> (	
.var	: Id ;	Exp8. <u>par</u> (<2>)	
.num	: IntConst ;	))))))))) ;	

(a) Java grammar fragment.

(b) *Abstract* syntax.(c) *Concrete* syntax.

Figure 3: Example specifying transformations using *abstract* vs. *concrete* syntax. (For emphasis, we have underlined the negation and parenthesis constructions.)

a subset of Java that deals with associativity and precedence of expressions by factorizing operators into several distinct levels according to operator precedence (as commonly found in programming language grammars); in this case, there are nine levels from `Exp` and `Exp1` all the way to `Exp8`.

Now suppose we were to *extend* the syntax of Java by adding a new statement, `repeat-until`, with syntax: `"repeat" Stm "until" "(" Exp ")" ";"`. Such a construction can easily be transformed into core Java by desugaring it into a `do-while` with a negated condition. Figure 3(b) shows how this would be done at the abstract syntactic level, using abstract syntax trees (ASTs). Transformation arguments are written in angled brackets; e.g., `<1>` and `<2>` (as explained later). Since negation is found at the eighth precedence level (in `Exp7`), the AST fragment for specifying the negated conditional expression would have to take us from `Exp` all the way to `Exp7`, add the negation “`Exp7.neg(...)`”, before adding the parentheses “`Exp8.par(...)`” and the second argument, “`<2>`” (which contains the original expression that was to be negated). Figure 3(c) specifies the same transformation, but at the concrete syntactic level, using strings instead of ASTs. At this level, there is no need for dealing explicitly with such low-level considerations which are more appropriately dealt with by the parser.

Interestingly, if the grammar of a language is unambiguous and we choose a canonical unparsing, we may move reversibly between abstract syntax trees and concrete syntactic program strings. Since we have such a recent ambiguity analysis [3], we have chosen to base the tool on concrete syntax. However, transformations may also be written in abstract syntax as in Figure 3(b).

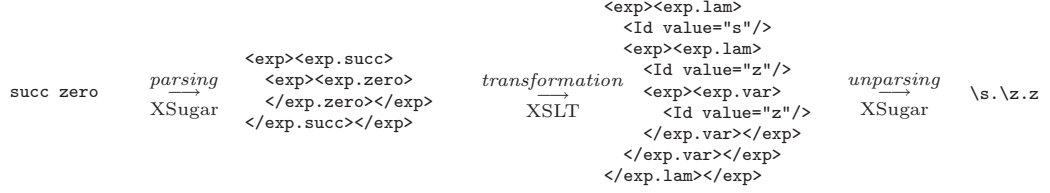


Figure 4: The transformation process.

5.2 Underlying technologies

Figure 4 depicts the transformation process. The Banana Algebra Tool is currently based on XSugar [5] and XSLT², but the tool is easily modified to use other underlying tools (only code generation is affected by these choices). We use XSugar for parsing a concrete term of the source language (e.g., “`succ zero`”) to an AST represented in XML. (XSugar uses an eager variant of Earley’s algorithm, capable of parsing any CFG, and a conservative ambiguity analysis [3] which may be used to verify unambiguity of all languages involved.) Then, we use XSLT for performing the catamorphic transformation from source AST to target AST. Finally, XSugar unparses the AST into an output term of the target language.

5.3 Other implementation issues

We found it convenient to permit lexical structure to be specified using regular expressions, as often encountered in parser/scanner tools. However, the tool currently considers this an atomic terminal layer that cannot be transformed.

We handle whitespace via permitting a special whitespace terminal named “\$” to be defined (it defaults to the empty regular expression). The semantics is that the whitespace is interspersed *between* all terminal and nonterminals on the right-hand-side of all productions. For embedded languages, it might be interesting to have finer grained control over this, but that is currently not supported by our tool.

In the future, it would be interesting to also add a means for *alpha conversion* and *static semantics* checks on top of the syntactic specifications

6 Examples and Evaluation

The tool can be used for any syntax-directed transformation that can be expressed as catamorphisms (which includes all the transformations of Metafront [7] and XSugar [5]). This includes translation between different languages, transformations on a language, and format conversion, but here we will focus on

²<http://www.w3.org/>

<pre> { \$ = [\n\t\r]* ; Id = [a-z]+ ; exp.var : Id ; exp.lam : "\\\" Id \".\" exp ; exp.app : "(" exp exp ")" ; } </pre>	<pre> <u>let</u> l = "lambda.1" <u>in</u> <u>let</u> ln = "lambda-num.1" <u>in</u> <u>letx</u> ln2l = (ln -> l [exp -> exp] exp.zero = '\z.z' ; exp.succ = '\s.<1>' ; exp.pred = '<1> \z.z' ;) <u>in</u> ln2l + <u>idx</u>(l) </pre>
--	---

(a) Language: λ -Calculus (with standard whitespace definition: “[\n\t\r]*”).

(b) Transformation: λ -Calculus extended with numerals to core λ -Calculus (cf. Fig 1).

Figure 5: Banana Algebra example programs: a *language* and a *transformation*.

language extension from each of the “four scenarios” from the introduction. Before that, however, we would like to show a concrete example program.

We will now revisit the example of extending the λ -Calculus with numerals that we have previously seen as a *catamorphism* (in Section 3.1) and later (in Figure 1) as a *general extension pattern*, motivating the algebraic approach.

Figure 5(a) shows the λ -Calculus as a Banana Algebra *language* constant (with standard whitespace, as defined by: “\$ = [\n\t\r]*”). Figure 5(b) defines the transformation from the λ -Calculus extended with numerals to the core calculus (cf., Figure 1). First, the contents of the file “lambda.1” (which we assume to contain the constant in Figure 5(a)) is loaded and bound to the Banana Algebra variable, *l* in the rest of the program. Then, in that program, *ln* is bound to the language containing the extension (assumed to reside in the file “lambda-num.1”). After this, *ln2l* is bound to the constant transformation that transforms the numeral extension to the core λ -Calculus. Finally, that constant transformation is added to *idx(l)* which is the identity transformation on the λ -Calculus.

Similarly, The Banana Algebra Tool can be used to extend Java with lots of syntactic constructions which can be desugared into Java itself; e.g., **for-each** control structures, enumeration declarations, design patterns templates, and so on. Here, we will give only one simple example of a Java extension; the **repeat-until** of Figure 3(c):

```

let java = "java.1"
in let repeat = { Stm.repeat : "repeat" Stm "until" "(" Exp ")" " "; }
in letx repeat2java =
  (| repeat -> java [Stm -> Stm, Exp -> Exp]
    Stm.repeat = 'do <1> while (!(<2>));' ;
  |)
in repeat2java + idx(java)

```

Although the Java grammar is big (“java.1” is a standard 575-line context-free grammar for Java), the **repeat-until** transformation is only seven lines.

More ambitiously, The Banana Algebra Tool may be used to embed entire DSLs into a host language. We have used the tool to embed standard SQL construc-

tions into the <bigwig> [4] language; e.g., the “**select-from-where**” construction may be captured by the following simple transformation:

```
stm.select = 'factor(<2>) { if (<3>) { return # \+ (<1>); } }' ;
```

Once defined, languages and transformations can all be added, composed, or otherwise put together. Thus, a programmer can use the tool to essentially tailor his own macro-extended language; e.g., “(java \ while) + sql”.

Relying on the existence of the tool, we have used the tool on itself to add more operators to the algebra. We can easily extend the Banana Algebra with an *overwrite* operator “<<” on languages and transformations (defined in terms of the core algebra):

$$\begin{aligned} \llbracket L_1 \ll L_2 \rrbracket_L &= (L_1 \setminus L_2) + L_2 \\ \llbracket X_1 \ll X_2 \rrbracket_X &= (X_1 \setminus \mathbf{src}(X_2)) + X_2 \end{aligned}$$

To put the algebraic and incremental development approach to the test, we have built an entire existing functional language “Fun” (used in an undergraduate course on teaching functional programming at Aarhus University and Aalborg University). The language extends the λ -Calculus with *arithmetic*, *lists*, *pairs*, *local definitions*, *numerals* in terms of arithmetic, *signed arithmetic* in terms of booleans and pairs, *fixed-point iterators* in terms of local definitions, *types* in terms of arithmetic and pairs. The entire language is specified incrementally using 245 algebraic operators (i.e., 58 constant languages, 51 language inclusions, 28 language additions, 23 language variables, 17 constant transformations, 17 transformation additions, 14 transformation inclusions, 10 local definitions, 9 identity transformations, 8 compositions, 4 language restrictions, 4 transformation variables, and 2 source extractions). The entire transformation reduces to a constant (constructive catamorphism) transformation of size 4MB. (For more on this transformation, we refer to [1].)

7 Related Work

Our work shares many commonalities and goals with that of *syntax macros*, *source transformation systems*, and *catamorphisms* (from a category theory perspective) the relation to which will be outlined below.

Syntax macros [6, 21] provide a means to *unidirectionally extend* a “host language” on top of which the macro system is hard-wired. Extension by syntactic macros corresponds to having control over only “step iii)” of Figure 1 (some systems also permit limited control over what corresponds to “step ii)”). By contrast, our algebraic approach can be used to extend the syntax of *any* language or transformation; and not just in one direction—extensions may be achieved through addition, composition, or otherwise modular assembly of other previously defined languages or transformations. Uni-directional extension is just one form of incremental definition in our algebraic approach.

The work on *extensible syntax* [9] improves on the definition flexibility in providing a way of defining grammars *incrementally*. However, it supports only three general language operations: extension, restriction, and update.

Compiler generator tools, such as Eli [12], Elan [2], Stratego/XT [8], ASF+SDF [18], TXL [10], JastAdd [13], and Silver [22] may all be used for source-to-target language transformation. They all have wider ambitions than our work, supporting specifications of full-scale compilers, many including static and dynamic semantics as well as Turing Complete computation on ASTs of the source language which obviously precludes our level of safety guarantees.

Although many of the tools support modular language development, none of them provide an algebra on top of their languages and transformations.

Systems based on *attribute grammars* (e.g., Eli, JastAdd, and Silver) may be used to indirectly express source-to-target transformations. This can be achieved through Turing Complete computation on the AST of the source language which compute terms of the target language in a downward or upward fashion (through *synthesized* and *inherited* attributes), or combinations thereof. In contrast, catamorphisms are restricted to upward inductive recombination of target ASTs. Our transformations could easily be generalized to also construct target AST downwards, by simply allowing catamorphisms to take target typed *AST arguments* (as detailed in [7], p. 17). This corresponds to a notion of *anamorphisms* and *hylomorphisms*, but would compromise compile-time elimination of composition (since anamorphisms and catamorphisms in general cannot be fused into one transformation, without an intermediate step).

Systems based on *term rewriting* (e.g., Elan, TXL, ASF+SDF, and Stratego/XT) may also be used to indirectly express source-to-target transformations. However, a transformation from language S to T has to be *encoded* as a rewriting working on terms of combined type: $S \cup T$ or $S \times T$. Although the tools may syntactically check that each rewriting step respects the grammars, the formalism comes with three kinds of termination problems which cannot be statically verified in either of the tools; a transformation may: i) never terminate; ii) terminate too soon (with unprocessed source terms); and, iii) be capable of producing a forest of output ASTs which means that is the responsibility of the programmer to ensure that the end result is one single output term. To help the programmer achieve this, rewriting systems usually offer control over the rewriting strategies.

In order to issue strong safety guarantees, in particular termination, we clearly sacrifice expressibility in that the catamorphisms are *not* able to perform Turing Complete transformations. However, previous work using constructive catamorphisms for syntactic transformations (e.g., Metafront [7] and XSugar [5]) indicate that they are sufficiently expressive and useful for a wide range of applications.

Of course, catamorphisms may be mimicked by disciplined style of functional programming, possibly aided by traversal functions automatically synthesized from datatypes [15], or by libraries of combinators [17]. However, since within a general purpose context, it cannot provide our level of safety guarantees and would not be able to compile-time factorize composition (although the functional techniques *deforestation/fusion* [20, 11, 19] may—in some instances—be used to achieve similar effects).

There exists a body of work on catamorphisms in a category theoretical

setting [14, 16]. However, these are theoretical frameworks that have not been turned into practical tool implementations supporting the notion of addition on languages and transformations which plays a crucial role in the extension pattern of Figure 1 and many of the examples.

8 Conclusion

The algebraic approach offers via 16 operators a *simple*, *incremental*, and *modular* means for specifying syntactic language extensions through algebraic composition of previously defined languages and transformations. The algebra comes “for free” in that any algebraic transformation term can be statically *reduced* to a constant transformation without compromising the strong *safety* and *efficiency* properties offered by catamorphisms.

The tool may be used by: 1) programmers to extend existing languages with their own macros; 2) developers to embed DSLs in host languages; 3) compiler writers to implement only a small core language (and specify the rest externally as extensions); and 4) developers and teachers to build multi-layered languages. The Banana Algebra Tool is available—as 3,600 lines of O’Caml code—along with examples from its homepage:

[<http://www.itu.dk/people/brabrand/banana-algebra/>]

Acknowledgments

The authors would like to acknowledge Kevin Millikin, Mads Sig Ager, Per Graa, Kristian Støvring, Anders Møller, Michael Schwartzbach, and Martin Sulzmann for useful comments and suggestions.

References

- [1] Jacob Andersen and Claus Brabrand. Syntactic language extension via an algebra of languages and transformations. ITU Technical Report. Available from: <http://www.itu.dk/people/brabrand/banana-algebra/>, 2008.
- [2] P. Borovansky, C. Kirchner, H. Kirchner, P. Moreau, and C. Ringeissen. An overview of elan. In *Second Intl. Workshop on Rewriting Logic and its Applications*, volume 15, 1998.
- [3] Claus Brabrand, Robert Giegerich, and Anders Møller. Analyzing ambiguity of context-free grammars. In *Proc. 12th International Conference on Implementation and Application of Automata, CIAA '07*, July 2007.
- [4] Claus Brabrand, Anders Møller, and Michael I. Schwartzbach. The <bigwig> project. *ACM Transactions on Internet Technology*, 2(2):79–114, 2002.

- [5] Claus Brabrand, Anders Møller, and Michael I. Schwartzbach. Dual syntax for XML languages. *Information Systems*, 33(4), June 2008. Earlier version in Proc. 10th International Workshop on Database Programming Languages, DBPL '05, Springer-Verlag LNCS vol. 3774.
- [6] Claus Brabrand and Michael I. Schwartzbach. Growing languages with metamorphic syntax macros. In *Proc. ACM SIGPLAN Workshop on Partial Evaluation and semantics-based Program Manipulation, PEPM'02*. ACM, 2002.
- [7] Claus Brabrand and Michael I. Schwartzbach. The metafront system: Safe and extensible parsing and transformation. *Science of Computer Programming Journal (SCP)*, 68(1):2–20, 2007.
- [8] Martin Bravenboer, Karl Trygve Kalleberg, Rob Vermaas, and Eelco Visser. Stratego/xt 0.17. a language and toolset for program transformation. *Science of Computer Programming*, 72(1-2):52–70, 2008.
- [9] Luca Cardelli, Florian Matthes, and Martin Abadi. Extensible syntax with lexical scoping. SRC Research Report 121, 1994.
- [10] J.R. Cordy. Txl - a language for programming language tools and applications. In *Proceedings of ACM 4th International Workshop on Language Descriptions, Tools and Applications (LDTA'04)*, pages 1–27, April 2004.
- [11] João Paulo Fernandes, Alberto Pardo, and João Saraiva. A shortcut fusion rule for circular program calculation. In *Haskell '07: Proceedings of the ACM SIGPLAN workshop on Haskell workshop*, pages 95–106. ACM, 2007.
- [12] Robert W. Gray, Steven P. Levi, Vincent P. Heuring, Anthony M. Sloane, and William M. Waite. Eli: a complete, flexible compiler construction system. *Communications of the ACM*, 35(2):121–130, 1992.
- [13] Görel Hedin and Eva Magnusson. Jastadd - a java-based system for implementing frontends. In *Electronic Notes in Theoretical Computer Science*, volume 44(2). Elsevier Science Publishers, 2001.
- [14] Richard B. Kieburtz and Jeffrey Lewis. Programming with algebras. In *Advanced Functional Programming, number 925 in Lecture Notes in Computer Science*, pages 267–307. Springer-Verlag, 1995.
- [15] R. Lämmel, J. Visser, and J. Kort. Dealing with Large Bananas. In J. Jeuring, editor, *Proceedings of WGP'2000, Technical Report, Universiteit Utrecht*, pages 46–59, July 2000.
- [16] Erik Meijer, Maarten Fokkinga, and Ross Paterson. Functional programming with bananas, lenses, envelopes and barbed wire. In J. Hughes, editor, *Proceedings 5th ACM Conf. on Functional Programming Languages and Computer Architecture, FPCA'91, Cambridge, MA, USA, 26–30 Aug 1991*, volume 523, pages 124–144. Springer-Verlag, Berlin, 1991.

- [17] S. Doaitse Swierstra, Pablo R. Azero Alcocer, and João Saraiva. Designing and implementing combinator languages. In *Third Summer School on Advanced Functional Programming, volume 1608 of LNCS*, pages 150–206. Springer-Verlag, 1999.
- [18] M. G. J. van den Brand, A. van Deursen, J. Heering, H.A. de Jong, M. de Jonge, T. Kuipers, P. Klint, L. Moonen, P. A. Olivier, J. Scheerder, J. J. Vinju, E. Visser, and J. Visser. The ASF+SDF meta-environment: a component-based language development environment. In *Proc. Compiler Construction 2001*. Springer-Verlag, 2001.
- [19] Janis Voigtländer. Semantics and pragmatics of new shortcut fusion rules. In Jacques Garrigue and Manuel Hermenegildo, editors, *Proc. Functional and Logic Programming*, volume 4989 of *LNCS*, pages 163–179. Springer-Verlag, April 2008.
- [20] Philip Wadler. Deforestation: Transforming programs to eliminate trees. *Theoretical Computer Science*, 73:344–358, 1990.
- [21] Daniel Weise and Roger F. Crew. Programmable syntax macros. In *Programming Language Design and Implementation (PLDI)*, pages 156–165, 1993.
- [22] Eric Van Wyk, Derek Bodin, Jimin Gao, and Lijesh Krishnan. Silver: an extensible attribute grammar system. *Electronic Notes in Theoretical Computer Science*, 203(2):103–116, 2008.

A Semantics of the algebra

We will now exploit the aforementioned self-containedness property and give a *big-step reduction semantics* for the algebra capable of reducing any language expression, L , to a constant language (context-free grammar), l ; and any transformation expression, X , to a constant transformation (constructive catamorphism), $x = \llbracket l_s \rightarrow l_t [\tau] c \rrbracket$.

Let EXP_L denote the set of all language expressions from the syntactic category, L ; and let EXP_X denote the set of all transformation expressions from the syntactic category, X . Also, we take VAR to be the set of all variables. We define environments in a straightforward way:

$$ENV_L : VAR \rightarrow EXP_L \qquad ENV_X : VAR \rightarrow EXP_X$$

The reduction semantics for the algebra of languages is defined by the relation $\Downarrow_L \subseteq ENV_L \times ENV_X \times EXP_L \times EXP_L$ (cf. Figure 6(a)). We will use the syntax “ $\alpha, \beta \vdash L \Downarrow_L l$ ” as a shorthand for “ $(\alpha, \beta, L, l) \in \Downarrow_L$ ”. Similarly, the reduction semantics for the algebra of transformations is defined by the relation $\Downarrow_X \subseteq$

$$\begin{array}{l}
[\text{CON}]_L \quad \frac{}{\alpha, \beta \vdash l \Downarrow_L l} \vdash_{wfl} l \\
[\text{VAR}]_L \quad \frac{}{\alpha, \beta \vdash v \Downarrow_L \alpha(v)} \\
[\text{RES}]_L \quad \frac{\alpha, \beta \vdash L \Downarrow_L l \quad \alpha, \beta \vdash L' \Downarrow_L l'}{\alpha, \beta \vdash L \setminus L' \Downarrow_L l \ominus_l l'} \\
[\text{ADD}]_L \quad \frac{\alpha, \beta \vdash L \Downarrow_L l \quad \alpha, \beta \vdash L' \Downarrow_L l'}{\alpha, \beta \vdash L + L' \Downarrow_L l \oplus_l l'} \quad l \sim_l l' \\
[\text{SRC}]_L \quad \frac{\alpha, \beta \vdash X \Downarrow_X \mathbb{Q}l_s \rightarrow l_t [\tau] c\mathbb{D}}{\alpha, \beta \vdash \mathbf{src} (X) \Downarrow_L l_s} \\
[\text{TGT}]_L \quad \frac{\alpha, \beta \vdash X \Downarrow_X \mathbb{Q}l_s \rightarrow l_t [\tau] c\mathbb{D}}{\alpha, \beta \vdash \mathbf{tgt} (X) \Downarrow_L l_t} \\
[\text{LET}]_L \quad \frac{\alpha, \beta \vdash L \Downarrow_L l \quad \alpha[v \mapsto l], \beta \vdash L' \Downarrow_L l'}{\alpha, \beta \vdash \mathbf{let} v=L \mathbf{in} L' \Downarrow_L l'} \\
[\text{LETX}]_L \quad \frac{\alpha, \beta \vdash X \Downarrow_X x \quad \alpha, \beta[w \mapsto x] \vdash L' \Downarrow_L l'}{\alpha, \beta \vdash \mathbf{letx} w=X \mathbf{in} L' \Downarrow_L l'}
\end{array}$$

(a) Semantics for the algebra of languages.

$$\begin{array}{l}
[\text{CON}]_X \quad \frac{\alpha, \beta \vdash L_s \Downarrow_L l_s \quad \alpha, \beta \vdash L_t \Downarrow_L l_t}{\alpha, \beta \vdash \mathbb{Q}L_s \rightarrow L_t [\tau] c\mathbb{D} \Downarrow_X \mathbb{Q}l_s \rightarrow l_t [\tau] c\mathbb{D}} \vdash_{wfx} \mathbb{Q}l_s \rightarrow l_t [\tau] c\mathbb{D} \\
[\text{VAR}]_X \quad \frac{}{\alpha, \beta \vdash w \Downarrow_X \beta(w)} \\
[\text{RES}]_X \quad \frac{\alpha, \beta \vdash X \Downarrow_X x \quad \alpha, \beta \vdash L \Downarrow_L l}{\alpha, \beta \vdash X \setminus L \Downarrow_X x \ominus_x l} \\
[\text{ADD}]_X \quad \frac{\alpha, \beta \vdash X \Downarrow_X x \quad \alpha, \beta \vdash X' \Downarrow_X x'}{\alpha, \beta \vdash X + X' \Downarrow_X x \oplus_x x'} \quad x \sim_x x' \\
[\text{COMP}]_X \quad \frac{\alpha, \beta \vdash X \Downarrow_X \mathbb{Q}l_s \rightarrow l_t [\tau] c\mathbb{D} \quad \alpha, \beta \vdash X' \Downarrow_X \mathbb{Q}l'_s \rightarrow l'_t [\tau'] c'\mathbb{D}}{\alpha, \beta \vdash X' \circ X \Downarrow_X \mathbb{Q}l_s \rightarrow l'_t [\tau' \circ \tau] c' \circ_c c\mathbb{D}} \quad l_t \sqsubseteq_l l'_s \\
[\text{IDX}]_X \quad \frac{\alpha, \beta \vdash L \Downarrow_L l}{\alpha, \beta \vdash \mathbf{idx} (L) \Downarrow_X \mathbb{Q}l \rightarrow l [id_\tau(l)] id_c(l) \mathbb{D}} \\
[\text{LET}]_X \quad \frac{\alpha, \beta \vdash L \Downarrow_L l \quad \alpha[v \mapsto l], \beta \vdash X' \Downarrow_X x'}{\alpha, \beta \vdash \mathbf{let} v=L \mathbf{in} X' \Downarrow_X x'} \\
[\text{LETX}]_X \quad \frac{\alpha, \beta \vdash X \Downarrow_X x \quad \alpha, \beta[w \mapsto x] \vdash X' \Downarrow_X x'}{\alpha, \beta \vdash \mathbf{letx} w=X \mathbf{in} X' \Downarrow_X x'}
\end{array}$$

(b) Semantics for the algebra of transformations.

Figure 6: Semantics of the algebra.

$ENV_L \times ENV_X \times EXP_X \times EXP_X$ (cf. Figure 6(b)). Again, we will use the short-hand syntax “ $\alpha, \beta \vdash X \Downarrow_X x$ ” instead of “ $(\alpha, \beta, X, x) \in \Downarrow_X$ ”.

Note that the reduction semantics in Figure 6 uses a range of operators ($\vdash_{wfl}, \sim_l, \oplus_l, \ominus_l, \sqsubseteq_l, \vdash_{wfx}, \sim_x, \oplus_x, \ominus_x, id_\tau, id_c$) which all operate on the level below that of the algebra; i.e., on constant languages (context-free grammars) and transformations (constructive catamorphisms). They can all be defined either at a *concrete* or *abstract* syntactic level. We refer to [1], for a formal specification of these lower-level operators in terms of abstract syntax.